Question 8: Record all Dockerfiles, Kubernetes manifests, commands, screenshots, and observations for each experiment.

This document compiles the recorded evidence of every Lab 2 experiment. Everything below is also committed in the GitHub repository under docs/.

1. How evidence was recorded

- Command sessions saved to docs/commands/*.log while working in the terminal
- Screenshots (terminal renders and Jenkins / Docker Hub web UI) in docs/screenshots/
- Per-experiment observations in docs/observations.md
- Everything committed and pushed with meaningful messages

2. Commit history (github.com/theabhinav0231/devops-lab-2)

```
* 62dcded add build 3 auto-trigger evidence and polling log screenshot
* 6cb9aa0 add q7 CD pipeline evidence and screenshots
* 496cf4e add lab 2 observations
* 0ff59ad fix CD health check: use kubectl node IP instead of minikube ip
* aeb0b2a add Jenkinsfile for continuous deployment to minikube
* eff439a add q5 k8s deployment and q6 rolling update, rollback, scaling evidence
* b8c283b add q2 container management, docker hub push and compose evidence
* 65e8eb8 add q1 docker build/run evidence, q3 storage and q4 network demos
* b8c283b (tag) initial commit: flask app v2.0.0 with Dockerfile, compose and k8s manifests
```

3. Dockerfile (Q1)

```
# Lab 2 - container image for the flask app
FROM python:3.12-slim

ARG APP_VERSION=2.0.0
ENV APP_VERSION=${APP_VERSION}

WORKDIR /app

# install dependencies first so this layer is cached
COPY app/requirements.txt /app/requirements.txt
RUN pip install --no-cache-dir -r /app/requirements.txt

# copy application code
COPY app/ /app/

EXPOSE 5000

# run as non-root user
RUN useradd -m appuser
USER appuser

CMD ["python", "app.py"]
```

4. Kubernetes manifests (Q5)

k8s/deployment.yaml

```
# Lab 2 - Deployment for the flask app
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lab2-app
  labels:
    app: lab2-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: lab2-app
  template:
    metadata:
      labels:
        app: lab2-app
    spec:
      containers:
        - name: lab2-app
          image: devops-lab2-app:2.0.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 5000
          envFrom:
            - configMapRef:
                name: lab2-app-config
          readinessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /health
              port: 5000
            initialDelaySeconds: 15
            periodSeconds: 20
```

k8s/service.yaml

```
# Lab 2 - NodePort service exposing the flask app
apiVersion: v1
kind: Service
metadata:
  name: lab2-app-service
spec:
  type: NodePort
  selector:
    app: lab2-app
  ports:
    - port: 5000
      targetPort: 5000
      nodePort: 30080
```

k8s/configmap.yaml

```
# Lab 2 - ConfigMap for the flask app
apiVersion: v1
kind: ConfigMap
metadata:
  name: lab2-app-config
data:
  APP_MESSAGE: "DevOps Lab 2 app running on Kubernetes (Minikube)"
```

5. Docker Compose (Q2)

docker/docker-compose.yml

```
# Lab 2 - multi-container setup: flask app + redis
services:
  web:
    build:
      context: ..
      dockerfile: docker/Dockerfile
    image: devops-lab2-app:2.0.0
    ports:
      - "5000:5000"
    environment:
      - APP_VERSION=2.0.0
    depends_on:
      - redis
    networks:
      - lab2net

  redis:
    image: redis:7-alpine
    container_name: lab2-redis
    ports:
      - "6379:6379"
    networks:
      - lab2net

networks:
  lab2net:
    driver: bridge
```

6. Key evidence screenshots

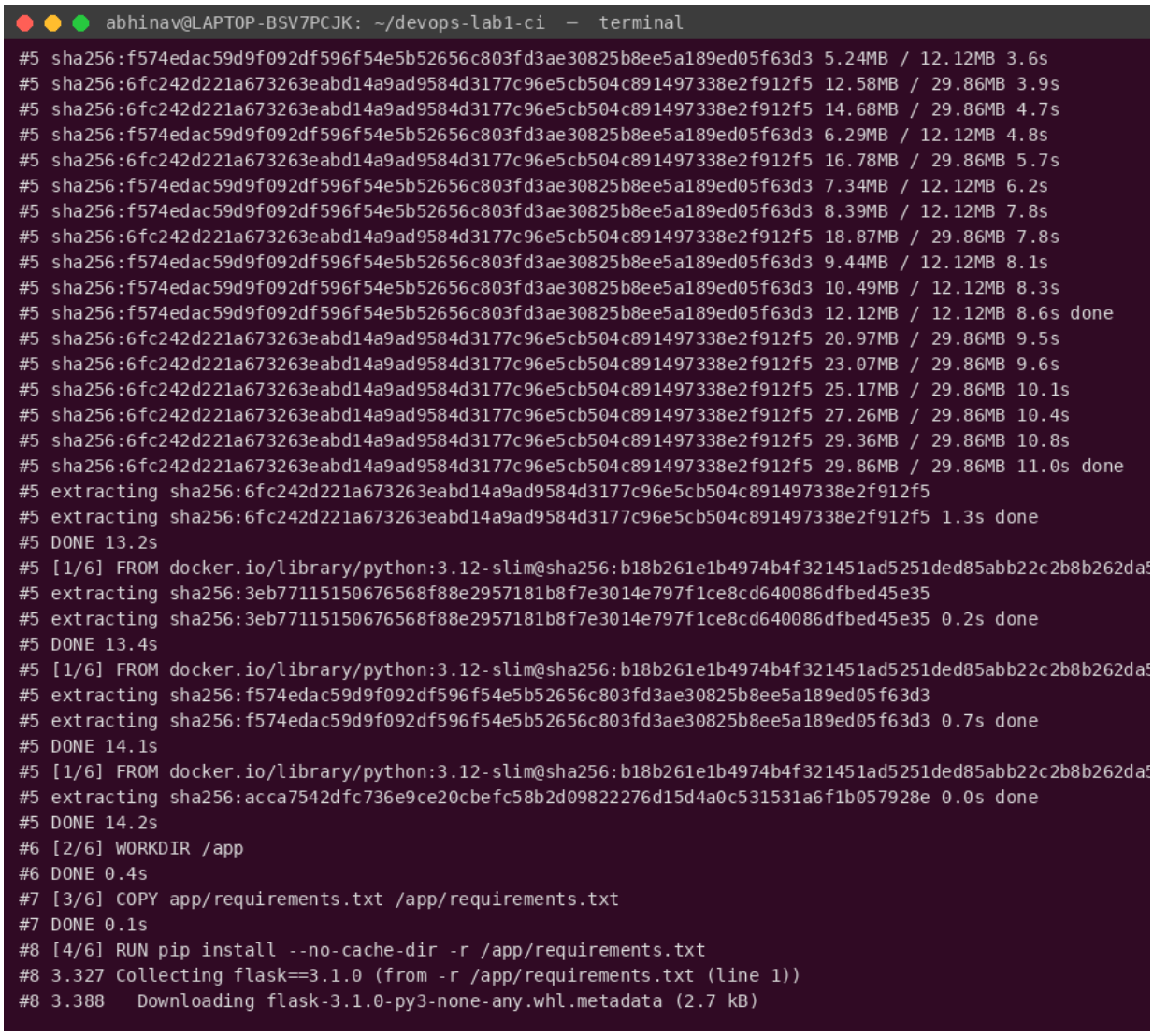


Fig: Q1: docker build and run

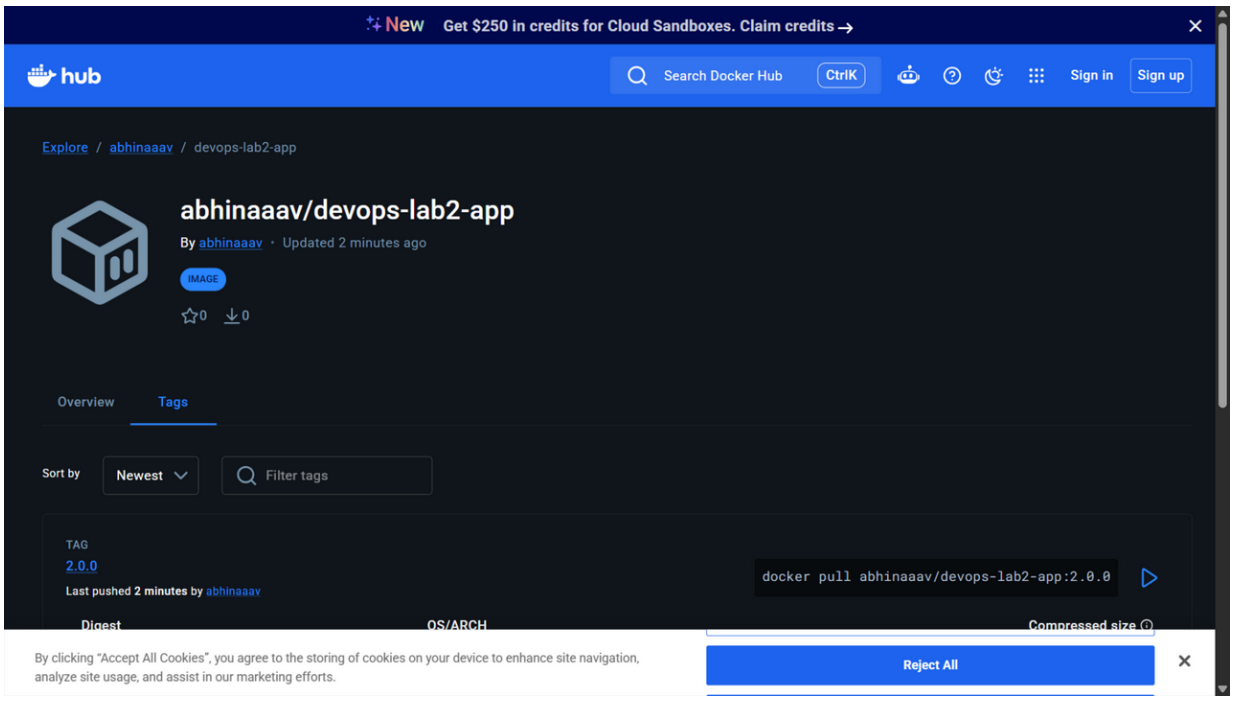


Fig: Q2: image pushed to Docker Hub

```
abhinav@LAPTOP-BSV7PCJK: ~/devops-lab1-ci  —  terminal
### Demo 2: a named volume SURVIVES container deletion
abhinav@LAPTOP-BSV7PCJK:~$ docker volume create lab2data
lab2data
abhinav@LAPTOP-BSV7PCJK:~$ docker run -d --name demo-vol -v lab2data:/data ubuntu:24.04 sleep infinity
c5e2183b5e8a4933f73199f9ef56a68767b0a923e3447e85ad75b03d3a07a625
abhinav@LAPTOP-BSV7PCJK:~$ docker exec demo-vol sh -c "echo persistent-data > /data/note.txt && cat /data/
persistent-data
abhinav@LAPTOP-BSV7PCJK:~$ docker rm -f demo-vol && docker run -d --name demo-vol -v lab2data:/data ubuntu
demo-vol
df34cc335ea2592ec4cffaf44142fdd4672dbcf9df3a0e9758cfe14633f3be93
abhinav@LAPTOP-BSV7PCJK:~$ docker exec demo-vol cat /data/note.txt
persistent-data
--> file SURVIVED: named volume lives outside the container lifecycle
abhinav@LAPTOP-BSV7PCJK:~$ docker volume inspect lab2data | grep -A2 Mountpoint
        "Mountpoint": "/var/lib/docker/volumes/lab2data/_data",
        "Name": "lab2data",
        "Options": null,
```

Fig: Q3: named volume demo

```
abhinav@LAPTOP-BSV7PCJK: ~/devops-lab1-ci  —  terminal
### Two containers talking over a user-defined bridge network
abhinav@LAPTOP-BSV7PCJK:~$ docker network create lab2net
abhinav@LAPTOP-BSV7PCJK:~$ docker network ls | grep lab2net
0f6f494d736a   lab2net    bridge    local
abhinav@LAPTOP-BSV7PCJK:~$ docker run -d --name net-a --network lab2net ubuntu:24.04 sleep infinity
78803bf67a504aeeb6d3423b4cebe4d317ae2c106fa97719ccbc546bbd3b5232
abhinav@LAPTOP-BSV7PCJK:~$ docker run -d --name net-b --network lab2net ubuntu:24.04 sleep infinity
245105f4700b21dff87ecd8fbbb3c42c150565d8c7e7c22257706ffcf0607049
abhinav@LAPTOP-BSV7PCJK:~$ docker exec net-a apt-get install -y -qq iputils-ping   (minimal image: add pi
ping installed in both containers
abhinav@LAPTOP-BSV7PCJK:~$ docker inspect net-a --format "net-a IP: {{.NetworkSettings.Networks.lab2net.IP
net-a IP: 172.18.0.2
abhinav@LAPTOP-BSV7PCJK:~$ docker inspect net-b --format "net-b IP: {{.NetworkSettings.Networks.lab2net.IP
net-b IP: 172.18.0.3
### Communication using the container NAME (embedded DNS)
abhinav@LAPTOP-BSV7PCJK:~$ docker exec net-a ping -c 2 net-b
PING net-b (172.18.0.3) 56(84) bytes of data.
64 bytes from net-b.lab2net (172.18.0.3): icmp_seq=1 ttl=64 time=9.50 ms
64 bytes from net-b.lab2net (172.18.0.3): icmp_seq=2 ttl=64 time=0.090 ms
--- net-b ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1007ms
rtt min/avg/max/mdev = 0.090/4.792/9.495/4.702 ms
### Communication using the container IP ADDRESS
abhinav@LAPTOP-BSV7PCJK:~$ docker exec net-a ping -c 2 172.18.0.3
PING 172.18.0.3 (172.18.0.3) 56(84) bytes of data.
64 bytes from 172.18.0.3: icmp_seq=1 ttl=64 time=0.087 ms
64 bytes from 172.18.0.3: icmp_seq=2 ttl=64 time=0.116 ms
--- 172.18.0.3 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1014ms
rtt min/avg/max/mdev = 0.087/0.101/0.116/0.014 ms
### Containers on the DEFAULT bridge network cannot resolve each other by name
abhinav@LAPTOP-BSV7PCJK:~$ docker run -d --name net-c ubuntu:24.04 sleep infinity
55e7620ea0cf21711585a65fd999571bdc9175eebdac7962bb405c4dd78faba7
abhinav@LAPTOP-BSV7PCJK:~$ docker exec net-c ping -c 1 net-a
ping: net-a: Name or service not known
--> default bridge has no embedded DNS: user-defined networks are required for name resolution
```

Fig: Q4: network demo

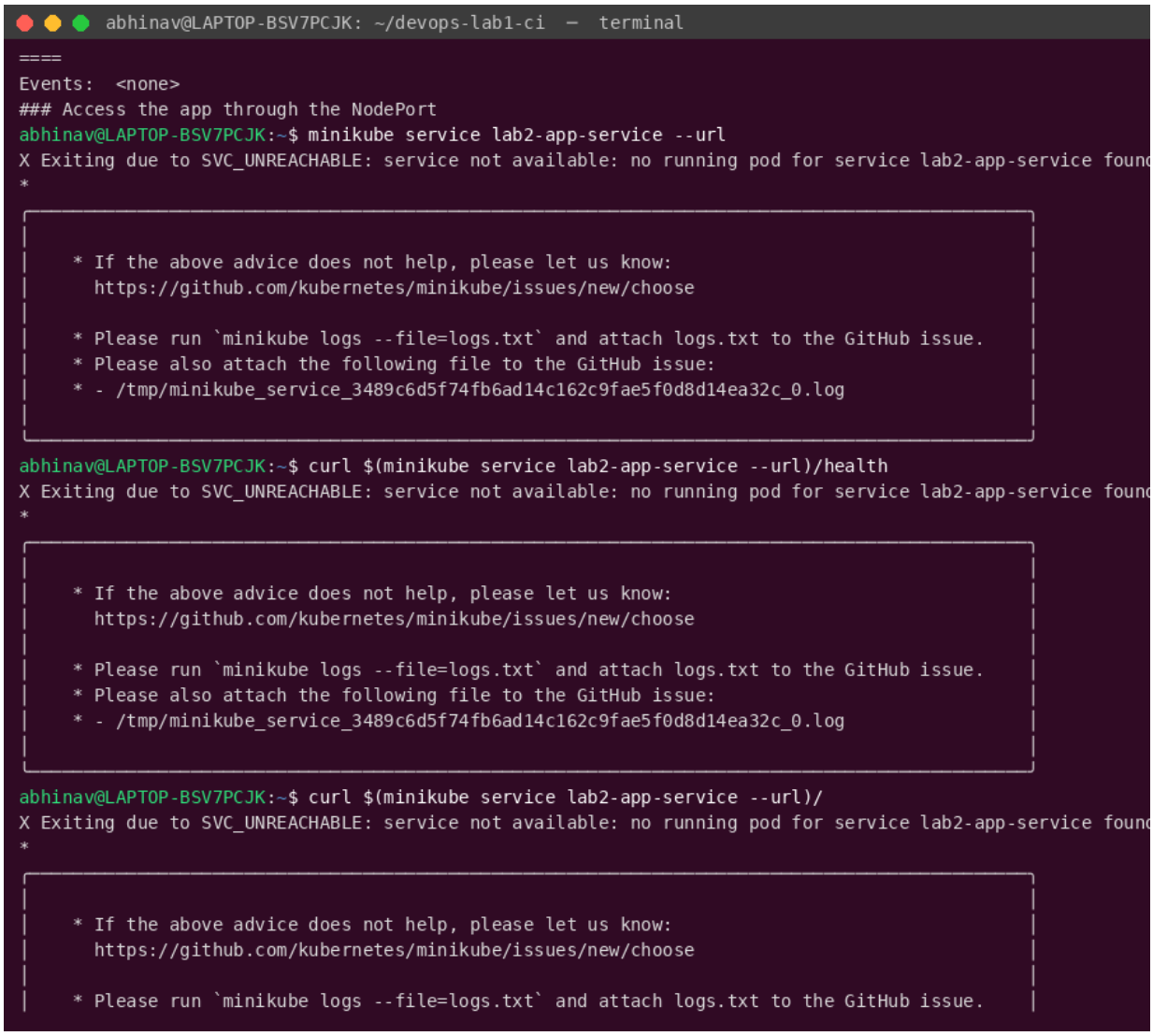


Fig: Q5: kubectl apply and pods

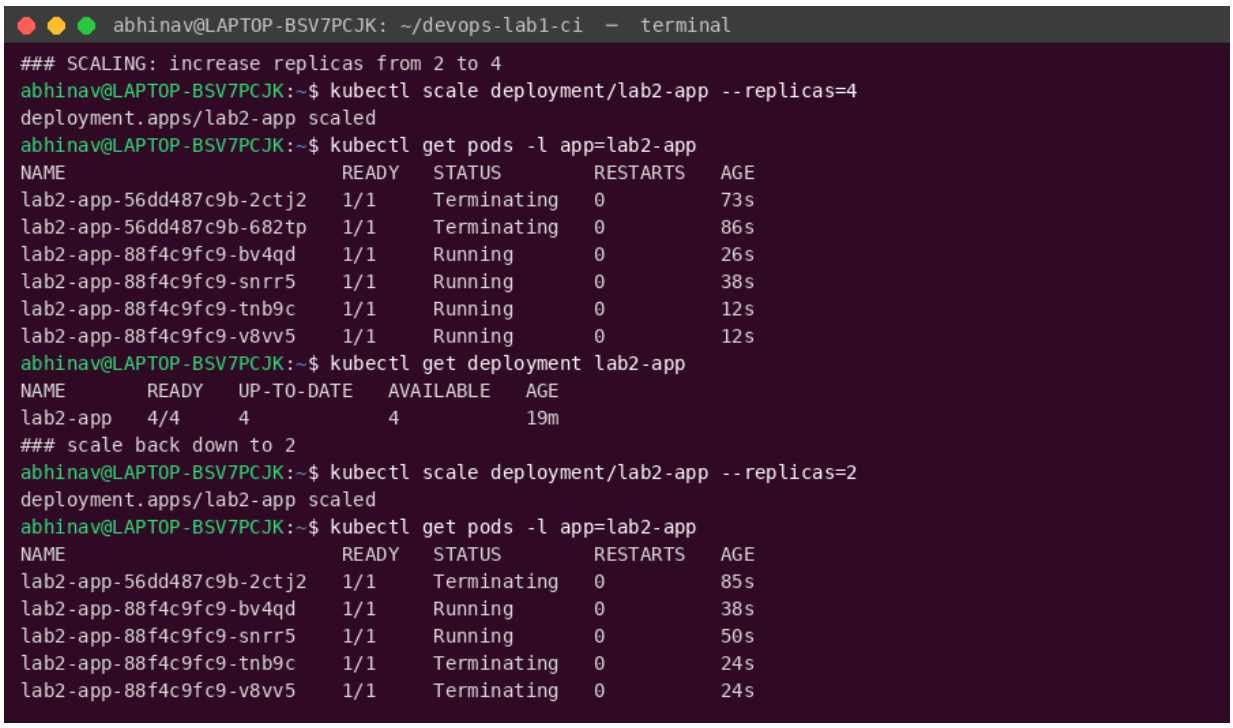


Fig: Q6: scaling demo

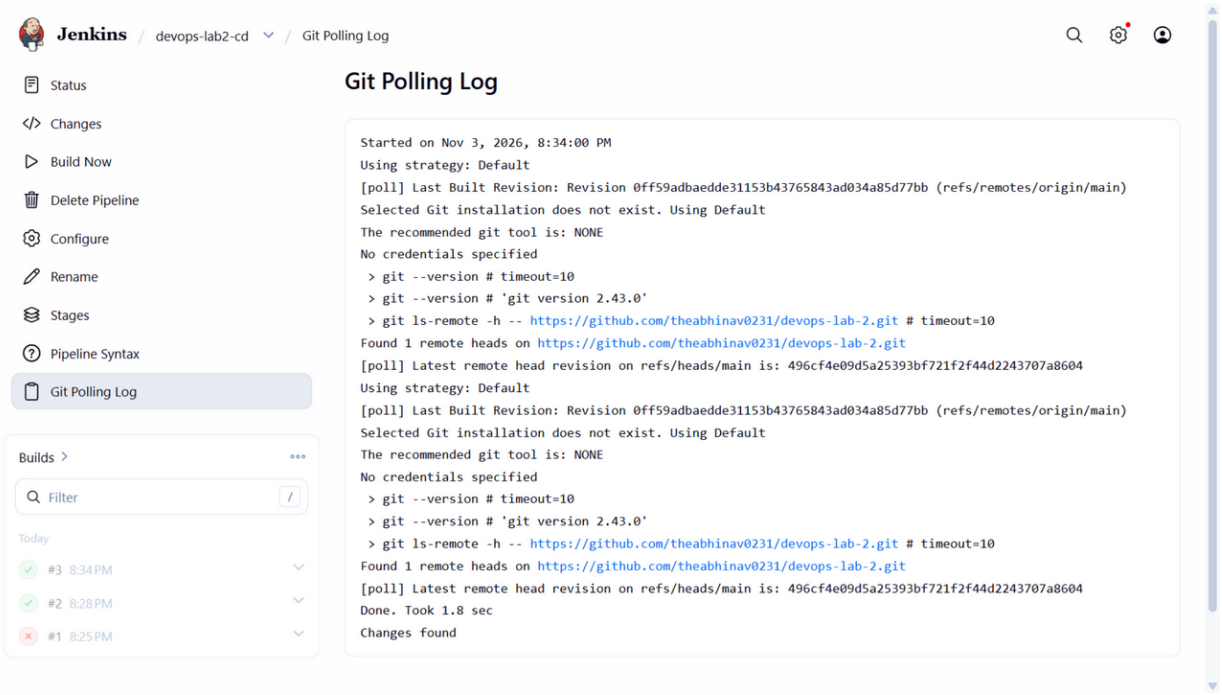


Fig: Q7: poll detected the push

7. Observations

```
# Lab 2 Observations

## Q1 - Containerize with Docker
- The image builds in layers: dependency install (`pip install`) was placed before the code copy so rebuilds
- `docker history` shows each layer with its size; the base image (python:3.12-slim) is most of the 204 MB,
- The container runs as a non-root user (`appuser`), which maps to uid 1000 (visible with `docker top`).
- Note: WSL2 + Docker 29.x uses the systemd cgroup driver by default, which breaks the minikube docker drive

## Q2 - Manage containers and images
- `docker stop` keeps the container (shows as Exited in `ps -a`); `start` resumes it. `rm` removes it entire
- Tagging with `docker tag` just adds a new name to the same image ID - `abhinaaav/devops-lab2-app:2.0.0` an
- Docker Hub accepted the push with a personal access token via `--password-stdin`; the image appeared on th
- Docker Compose: `up -d` created its own network (`docker_lab2net`), both services resolved each other by n
- Gotcha: the compose `web` service failed to start while the standalone container from Q1 still held port 5

## Q3 - Docker storage
- Container-layer data (`/tmp/note.txt`) disappeared after `docker rm` + recreate: writable layers die with
- A named volume (`lab2data`) survived deletion and recreation because it lives in `/var/lib/docker/volumes/
- A bind mount (`~/lab2-bindmount-data:/data`) synced both ways: host edits visible in the container, contai

## Q4 - Docker network
- On a user-defined bridge network, containers resolve each other by **name** through Docker's embedded DNS
- On the default `bridge` network there is no embedded DNS - `ping net-a` failed with "Name or service not k
- The minimal `ubuntu:24.04` image ships without `ping`; `apt install iputils-ping` was needed inside the de

## Q5 - Deploy to Kubernetes (Minikube)
- Minikube v1.39 on WSL2 needed two fixes: `--force-systemd=true` (minikube issue #20469 - without it the no
- With the containerd runtime, `eval $(minikube docker-env)` builds are experimental and failed (buildkit 40
- Deployment (2 replicas with readiness/liveness probes on `/health`) + NodePort Service (30080) + ConfigMap
- NodePort access works from inside WSL via the node IP (192.168.49.2:30080); from Windows, `kubectl port-fo

## Q6 - Rolling update, rollback, scaling
- `kubectl set image` triggered a rolling update: new pods came up one-by-one while old ones terminated - th
- `kubectl rollout undo` returned to the previous image; the version reverted from 2.0.1 to 2.0.0 because th
- Observation: the health curl taken *immediately* after `rollout undo` still returned 2.0.1 for a few secon
- `kubectl scale --replicas=4` produced 2 extra Running pods within seconds; scaling back to 2 terminated th

## Q7 - Continuous deployment
- The Jenkins user needed docker group membership and a copy of the kubeconfig (with cert paths rewritten in
- First CD run failed only at the final health check (the minikube profile issue above); build/push/deploy h
- A push to `main` is picked up by Poll SCM (H/5) and automatically: builds image `abhinaaav/devops-lab2-app
```